



DIPLOMADO

Desarrollo de sistemas con tecnología Java

Módulo7

Persistencia con Spring Data

M. en C. Jesús Hernández Cabrera

jesushernandezls7@aragon.unam.mx



Convenios

- Tolerancia de inicio de clase 15 min
- 20 minutos de receso

Evaluación

- Prácticas 30%
- Ejercicios 40%
- Avance proyecto 30%

Objetivo

- El participante será capaz de realizar la persistencia de datos a nivel avanzado a través del uso de Spring Data.

Objetivos particulares

- El alumno implementará aplicaciones usando el patrón de diseño Template y Callback.
- El alumno implementará aplicaciones con el patrón de diseño Repository de SpringData.
- Aprenderá la estructura y uso de los componentes de String Data.



DGTIC UNAM
DIRECCIÓN GENERAL DE CÓMPUTO Y
DE TECNOLOGÍAS DE INFORMACIÓN
Y COMUNICACIÓN



REDEC UNAM

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
DIRECCIÓN GENERAL DE CÓMPUTO Y DE TECNOLOGÍAS
DE INFORMACIÓN Y COMUNICACIÓN



DIPLOMADO

Desarrollo de sistemas con tecnología Java

Spring JDBC

Objetivo

- Identificar como Spring maneja la persistencia, por medio JDBC directo y ORM, empezando en de un nivel básico.
- Escribir un proyecto básico que implemente JDBC de medio nivel para entender la complejidad que reduce Spring data.

Contexto

- Spring JDBC es un módulo del núcleo de Spring Framework.
- Proporciona una capa de abstracción sobre JDBC puro.
 - Manejo manual de conexiones.
 - Gestión de excepciones.
 - Código repetitivo y propenso a errores.
 - Cierre de recursos (result sets, statements, conexiones).
- Simplifica las operaciones de base de datos en Java.
- No confundir con Spring Data JDBC.

Spring JDBC

- **Implementa dos patrones:** Template y Callback.
- **JdbcTemplate:** Clase central que ofrece métodos para consultas y actualizaciones.
- **Manejo de Excepciones:** Convierte las excepciones de SQL en excepciones de tiempo de ejecución específicas de Spring.
- **Control de recursos:** Gestiona automáticamente la apertura y cierre de recursos.
- **Mapeo de Objetos:** Mapea fácilmente filas de result sets a objetos.

<https://www.baeldung.com/spring-framework-design-patterns>

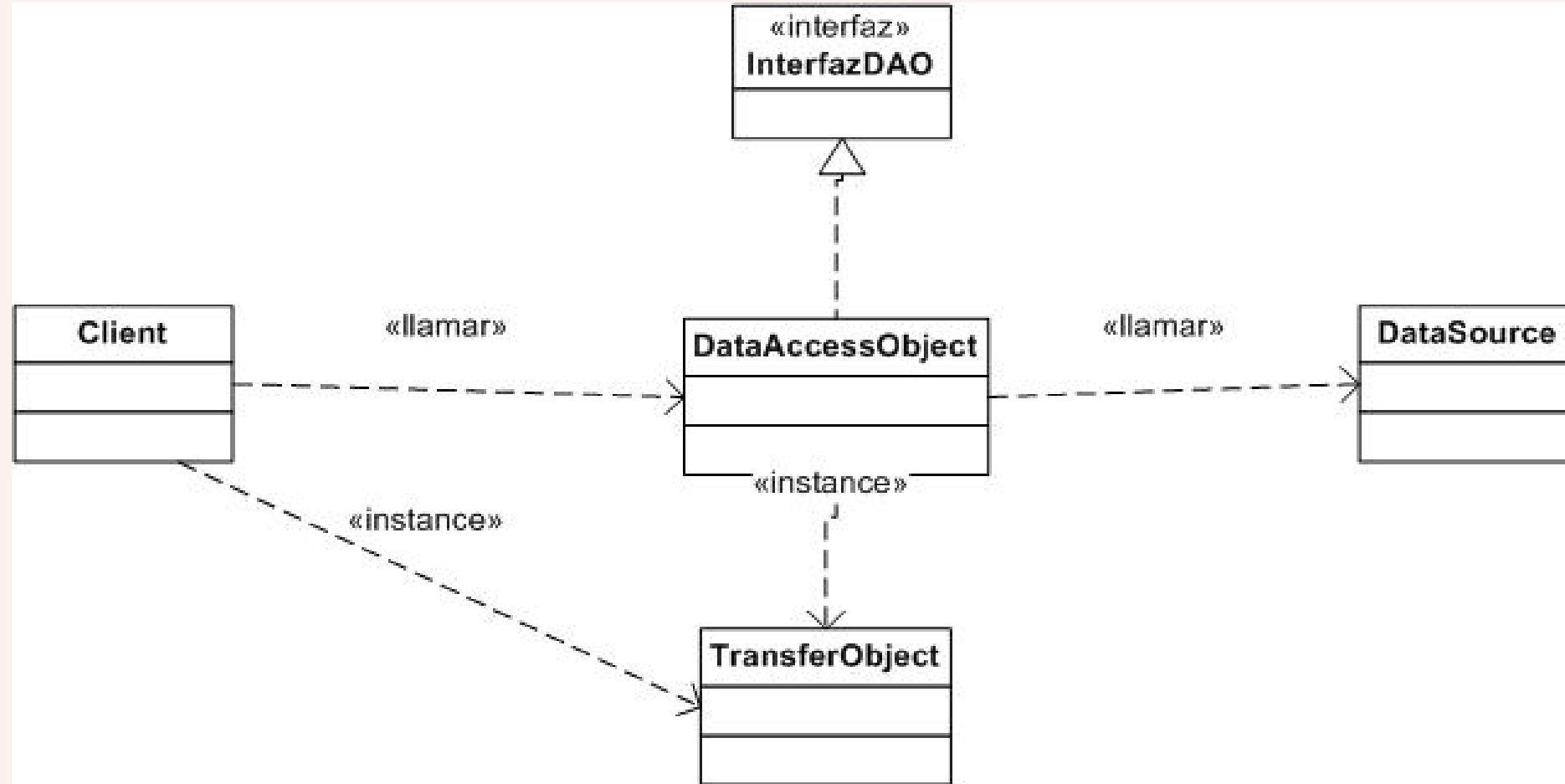
¿Cuándo se ocupa JDBC Template?

- La estructura de la base de datos no se puede modificar y que no este normalizada. Puedes escribir SQL Personalizado para manejar casos complejos.
- Tablas sin llave primaria (requeridos por los patrones de acceso a datos).
- Ocupar directamente SQL para aprovechar las funcionalidades que nos de la base de datos misma, logrando así un rendimiento optimo, por ejemplo uso constante de procedimientos almacenados.

Filosofía de acceso a datos en Spring Data

- Para obtener una persistencia de datos real, ocuparemos una de las estrategias que ocupa como patrón de diseño Java EE, estamos hablando del patrón DAO (Data Access Object, Objeto de Acceso a Datos).
- La idea básica es encapsular la creación, lectura, actualización y eliminación conocidas como funciones CRUD (Create, Read, Update, Delete) para trabajar con objetos y base de datos.
- Estas funcionalidades se hacen por medio de una interfaz, haciendo que el resto de la aplicación acceda por medio de ellas.
- **Nota:** Hay patrones mejores que DAO.

Arquitectura del patrón DAO.



Pasos para desarrollar una App con Spring JDBC

- 0.- Diseñar o estudiar el esquema de base de datos.
- 1.- Agregar Dependencias en Maven.
- 2.- Configurar la Fuente de Datos.
- 3.- Crear un DAO (Data Access Object)
- 4.- Utiliza JdbcTemplate para realizar operaciones CRUD (Crear, Leer, Actualizar).
- 5.- Manejo de Excepciones.
- 6.- Ejecutar instrucciones SQL.
- 7.- Procesar datos.
- 8.- Terminar conexiones.

Crear Esquema de base de datos de trabajo.

persona	
123	idpersona
ABC	nombre
123	edad

```
[MariaDB [basedgtic]> select * from persona;
+-----+-----+-----+
| idpersona | nombre | edad |
+-----+-----+-----+
|         1 | Rebeca |   34 |
|         2 | Samuel |   18 |
|         3 | Pedro  |   29 |
|         4 | Marcos |   22 |
|         5 | Maria  |   40 |
|         6 | Jose   |   30 |
|         7 | Manuel |   21 |
|         8 | Tomas  |   40 |
+-----+-----+-----+
8 rows in set (0.001 sec)

MariaDB [basedgtic]>
```

Crear el proyecto

Project
☐ Gradle - Groovy ☐ Gradle - Kotlin ☒ **Java** ☐ Kotlin ☐ Groovy
☒ **Maven**

Spring Boot
☐ 3.2.0 (SNAPSHOT) ☐ 3.2.0 (M3) ☐ 3.1.5 (SNAPSHOT) ☒ **3.1.4**
☐ 3.0.12 (SNAPSHOT) ☐ 3.0.11 ☐ 2.7.17 (SNAPSHOT) ☐ 2.7.16

Project Metadata

Group

Artifact

Name

Description

Package name

Packaging ☒ **Jar** ☐ War

Java ☐ 21 ☒ **17** ☐ 11 ☐ 8

Dependencies

JDBC API **SQL**
Database Connectivity API that defines how a client may connect to a database.

MariaDB Driver **SQL**
MariaDB JDBC and R2DBC driver.

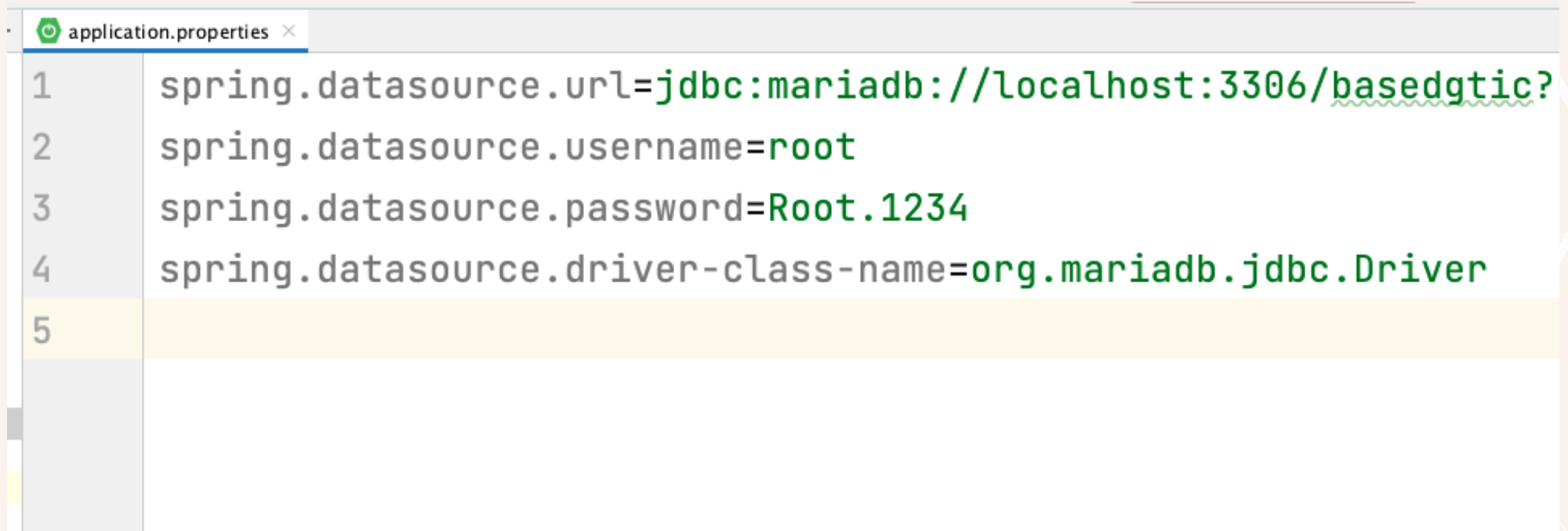
Agregar dependencias

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-jdbc</artifactId>
  </dependency>

  <dependency>
    <groupId>org.mariadb.jdbc</groupId>
    <artifactId>mariadb-java-client</artifactId>
    <scope>runtime</scope>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>
```

Configurar fuente de datos.

- application.properties



```
1 spring.datasource.url=jdbc:mariadb://localhost:3306/basedgtic?
2 spring.datasource.username=root
3 spring.datasource.password=Root.1234
4 spring.datasource.driver-class-name=org.mariadb.jdbc.Driver
5
```

Clase de modelo Persona.

```
import java.io.Serializable;

public class Persona implements Serializable {
    private static final long serialVersionUID = 1L;
    private Integer idPersona;
    private String nombre;
    private Integer edad;

    public Persona() {}
}
```

Crear un DAO (Data Access Object)

```
public interface PersonaDao {  
    public List<Persona> listarPersonas();  
    public Persona buscarPersonaPorId(Integer idPersona);  
    public Integer insertarPersona(Persona persona);  
    public Integer actualizarPersona(Persona persona);  
    public Integer eliminarPersona(Integer idPersona);  
    public Integer procEdadPersona(Integer idPersona);  
}
```

Utiliza JdbcTemplate para implementar la interfaz DAO.

```
@Repository
public class JdbcPersonaRepositoryImpl implements PersonaDao {
    private JdbcTemplate jdbc;

    public JdbcPersonaRepositoryImpl(JdbcTemplate jdbc) {
        super();
        this.jdbc = jdbc;
    }

    @Override
    public List<Persona> listarPersonas() {...}

    private Persona mapRowToPersona(ResultSet rs, int row) throws SQLException {...}

    @Override
```

Utiliza JdbcTemplate para implementar la interfaz DAO(Cont..)

```
@Override
public List<Persona> listarPersonas(){
    return jdbc.query("SELECT * from persona", this::mapRowToPersona);
    //return List.of();
}

private Persona mapRowToPersona(ResultSet rs, int row) throws SQLException {
    Persona pr = new Persona();
    pr.setIdPersona(rs.getInt(1));
    pr.setNombre(rs.getString(2));
    pr.setEdad(rs.getInt(3));
    return pr;
}
```

Ejecutar instrucciones SQL

```
@SpringBootApplication
public class Jdbcdemo2025Application implements CommandLineRunner {

    @Autowired
    private PersonaDao personaRepo;

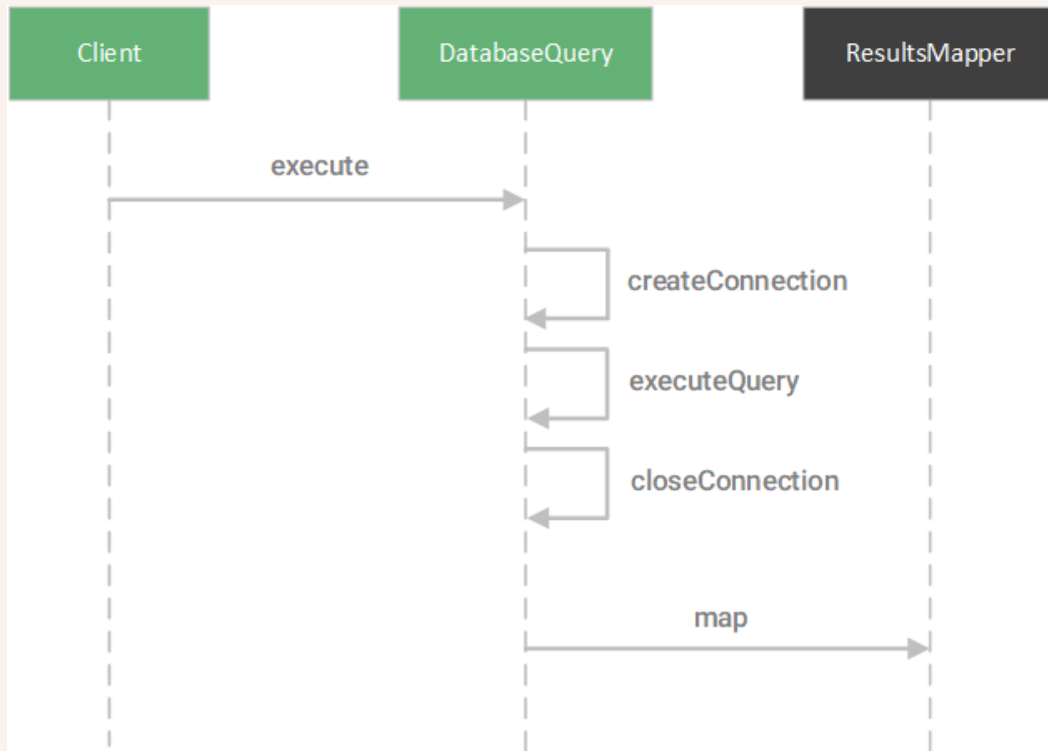
    public static void main(String[] args) {
        SpringApplication.run(Jdbcdemo2025Application.class, args);
    }

    @Override
    public void run(String... args) throws Exception {
        // SELECT
        for (Persona persona : personaRepo.listarPersonas() ) {
            System.out.println(persona.toString());
        }
    }
}
```


Beneficios de JDBC Template.

- JDBC Template implementa el patrón de Método de Plantilla, en donde se define el esqueleto de un proceso, en donde el proceso delega su trabajo a una interfaz para que ella cumpla los detalles específicos de implementación.
- Con esto Spring separa las partes fijas y variables del proceso de acceso a datos en dos clases distintas.

Beneficios de JDBC Template y su funcionamiento.



<https://www.baeldung.com/spring-framework-design-patterns>

Las clases de plantilla de los DAO de Spring se encargan de las tareas comunes de acceso a datos, y las tareas de las reglas de negocio se llama al objeto Retrollamada DAO personalizado.

JDBC Template ó ORM.

- Hoy día para la explotación de datos de una base de datos relacional se ocupa mucho las herramientas de mapeo ORM.



- Pero existen situaciones en donde no es viable ocupar una de estas herramientas, como única solución.

Contacto

M. en C. Jesús Hernández Cabrera
Profesor de carrera

jesushc@unam.mx

Redes sociales:



www.linkedin.com/in/hcjesus